

Serving-Aware Dynamic Batching as a Mixed-Precision Research Area

Runtime-Adaptive Mixed-Precision Quantization for LLM Inference

Prepared for the Runtime Adaptive Mixed Precision Quantization project

2026-07-08

Contents

Abstract	3
Executive summary	3
1. Problem framing: from local precision to shared precision	4
1.1 Why LLM inference makes mixed precision a systems problem	4
1.2 Why dynamic batching becomes the main focus	4
1.3 Definition: precision-aware dynamic batching	5
2. Background: mixed precision methods and what they miss	5
2.1 Uniform quantization	5
2.2 Static outlier-aware quantization	6
2.3 Static mixed precision	6
2.4 Any-precision and nested representations	6
2.5 Runtime adaptive methods	6
2.6 Continuous batching and serving systems	7
3. Taxonomy and comparison	7
3.1 Method comparison	7
3.2 Why dynamic batching is different from prior mixed precision	8
3.3 Relationship to soft pruning	8
4. Proposed research design: serving-aware dynamic batching	9
4.1 High-level architecture	9
4.2 Precision profile representation	9
4.3 Compatibility distance	10
4.4 Batch profile composition policies	10
Max profile	10
Mean or rounded profile	10
Quantile profile	10
Water-filling profile	11
Lane split	11
4.5 Offline trace collection	11
4.6 Profile discovery and compilation	11
4.7 Online scheduling policy	12
4.8 Runtime fallback	12
5. Strengths of dynamic batching as mixed precision	12
5.1 It attacks the systems bottleneck that local quantizers expose	12

5.2 It improves cache and memory residency	13
5.3 It reduces kernel and CUDA graph fragmentation	13
5.4 It aligns precision with QoS	13
5.5 It prevents easy requests from being dragged into worst-case precision	13
5.6 It creates a research bridge between quantization and serving	13
6. Weaknesses and risks	13
6.1 Queueing delay can erase latency gains	13
6.2 Profile prediction can be wrong	13
6.3 Shared profiles can waste bits	14
6.4 Too many profiles create fragmentation	14
6.5 Fine-grained token masks can be hardware-unfriendly	14
6.6 Interaction with KV cache is underdeveloped	14
6.7 Evaluation is harder than single-request quantization	14
7. Bottlenecks and suggested solutions	14
7.1 CPU-GPU and GPU-GPU transfer bottleneck	14
7.2 Irregular memory layout and bit packing	15
7.3 Kernel switching and launch overhead	15
7.4 Selector overhead	15
7.5 Queue fragmentation	15
7.6 Accuracy cliff and autoregressive error propagation	16
7.7 Prefill/decode asymmetry	16
7.8 KV-cache precision and memory pressure	16
7.9 Multi-GPU communication	16
7.10 Lack of benchmarks	16
8. Evaluation plan	17
8.1 Baselines	17
8.2 Metrics	17
Quality metrics	17
Serving metrics	17
Scheduler metrics	18
8.3 Workloads	18
8.4 Ablations	18
8.5 Hypotheses	18
9. Implementation roadmap	19
9.1 Stage 0: Trace-driven simulator	19
9.2 Stage 1: Multi-engine prototype	19
9.3 Stage 2: Nested representation executor	19
9.4 Stage 3: Continuous precision-aware batching	19
9.5 Stage 4: Compiler and hardware co-design	19
10. Detailed design choices	20
10.1 Profile codebook construction	20
10.2 Request difficulty prediction	20
10.3 Layer sensitivity prediction	20
10.4 Token fallback design	21
10.5 Memory policy	21
11. How this compares with other approaches	21
11.1 Compared with uniform PTQ	21
11.2 Compared with static outlier-aware methods	21
11.3 Compared with static mixed precision	22
11.4 Compared with QAQ	22
11.5 Compared with DP-LLM	22

11.6 Compared with MoBiQuant	22
11.7 Compared with ordinary continuous batching	22
12. Future research directions	23
12.1 Precision-aware continuous batching in production engines	23
12.2 Joint weight and KV-cache precision	23
12.3 Disaggregated prefill/decode scheduling	23
12.4 Safety-aware dynamic precision	23
12.5 Multi-tenant fairness and pricing	23
12.6 Hardware support for nested precision	24
12.7 Learned schedulers with safety constraints	24
12.8 Standard benchmark for runtime mixed precision serving	24
13. Recommended thesis direction	24
14. Conclusion	25
References	25

Abstract

Dynamic batching should be treated as a core mixed-precision research problem, not merely as a serving-system optimization. Existing mixed-precision quantization methods typically choose precision at one of three local granularities: request or query level, layer and decoding-step level, or token level. These methods can reduce memory traffic or preserve quality for individual requests, but a real LLM serving system executes many heterogeneous requests together. If every request independently chooses its own bit-widths, bit-planes, residual slices, or high-precision fallback blocks, the system can lose the gains through weight movement, kernel switching, poor CUDA graph reuse, irregular memory layouts, and fragmented batches.

This report frames dynamic batching as the missing serving-level layer in runtime adaptive mixed precision. The central idea is to predict a precision profile for each request, then group compatible requests so that a batch can share a block-level or layer-group precision plan while still allowing token-level fallbacks for risky cases. This turns precision from a local compression setting into a shared serving-time resource. The research challenge is to preserve the accuracy benefits of adaptive precision while making the execution path regular enough for high-throughput GPU serving.

The report compares dynamic batching with uniform quantization, static outlier-aware methods, Hessian or salience-based static mixed precision, any-precision quantization, and recent runtime-adaptive methods such as QAQ, DP-LLM, and MoBiQuant. It then gives a detailed design for precision-aware batching, identifies strengths and weaknesses, analyzes bottlenecks, proposes system and algorithmic solutions, and lays out future research directions.

Executive summary

The main thesis is:

Dynamic batching is the serving-level control plane for mixed precision. It decides not only how many requests should be executed together, but which requests can share the same precision path without wasting bits or causing unsafe accuracy loss.

Current adaptive mixed-precision methods answer local questions:

- Query-adaptive methods ask: how many bit-planes should this request use?
- Layer-adaptive methods ask: which layer needs high precision at this decoding step?
- Token-adaptive methods ask: which token should activate extra residual bit slices?

A serving system must answer a different question:

Which requests should share one batched execution path, given their predicted precision profiles, QoS deadlines, memory budget, and fallback risk?

The proposed research area is therefore **precision-aware dynamic batching**. It extends continuous or in-flight batching with precision compatibility. Instead of batching only by arrival time, sequence length, or KV-cache availability, the scheduler also uses a compact representation of each request’s expected precision demand.

A useful precision profile has four parts:

1. **Request budget:** scalar target, such as average 3.5-bit, memory budget, or latency target.
2. **Block vector:** which transformer blocks or layer groups need higher precision.
3. **Token fallback policy:** when uncertain tokens may activate extra residual slices or recompute high-risk operations.
4. **Memory and prefetch plan:** which bit-planes, residual slices, or weight pages should already be in GPU memory.

The strongest potential contribution is not another quantizer. It is a scheduler-executor co-design that makes adaptive precision batch-compatible. The scheduler trades a small amount of queueing delay for fewer distinct precision profiles, fewer CPU-GPU transfers, fewer kernel switches, more cache reuse, and better tail latency under an explicit accuracy budget.

1. Problem framing: from local precision to shared precision

1.1 Why LLM inference makes mixed precision a systems problem

LLM inference is often memory-bound during autoregressive decoding. At low batch size, each generated token repeatedly streams large weight matrices and accesses a growing KV cache. Quantization reduces the number of bits moved per weight or activation, but the practical speedup depends on whether the hardware can execute the lower precision path efficiently. A nominal 4-bit model does not automatically become twice as fast as an 8-bit model if dequantization, packing, kernel launch overhead, or memory movement dominates.

Mixed precision tries to spend bits only where they matter. The attractive idea is simple: fragile components receive more precision; insensitive components receive fewer effective bits. The hard part is that the definition of fragile is not fixed. It can depend on the prompt, the decoding step, the layer, the token, the activation outlier pattern, the current batch, and hardware state.

The user-provided project deck motivates this as a shift from hard pruning to soft pruning. Instead of removing model components with a binary 0/1 decision, the system keeps information accessible at different precisions. The research question becomes: which components deserve more bits for this input, and when is the hardware cost of those bits worth paying?

1.2 Why dynamic batching becomes the main focus

A single request can make a fine-grained precision decision. A serving system cannot afford unlimited fine-grained heterogeneity inside the same batch. GPUs prefer regular memory layouts, aligned loads, predictable kernels, and large enough batched GEMMs. If every request in a batch wants a different layer precision vector, a naive executor has three bad options:

1. **Use the maximum precision required by any request.** This protects accuracy but wastes bits on easy requests.
2. **Split requests into many small precision-specific batches.** This preserves adaptivity but hurts batch utilization and tail latency.
3. **Switch kernels and load weight slices per request inside a batch.** This preserves both local choices and grouping, but can destroy throughput through irregular execution.

Dynamic batching is the natural place to resolve this conflict. It can group requests by compatibility and select one shared precision plan per batch or microbatch. The scheduler can still preserve local

adaptivity by allowing a small fallback mechanism for outlier tokens or high-risk requests.

1.3 Definition: precision-aware dynamic batching

A standard dynamic batcher groups requests that arrive within a short window. A continuous or in-flight batcher forms a new batch at each decoding iteration, returning completed requests and admitting new ones. A **precision-aware dynamic batcher** extends this with precision compatibility.

Let request r at decoding step t have a predicted precision profile:

$$\pi(r, t) = (b_r, p_{r,1:L}(t), u_r(t), m_r(t), d_r)$$

where:

- b_r is a request-level precision or latency budget.
- $p_{r,1:L}(t)$ is a vector of desired layer or block precisions.
- $u_r(t)$ is an uncertainty or risk score.
- $m_r(t)$ is a memory/prefetch requirement, such as bit-planes or residual slices.
- d_r is the request deadline or QoS class.

The batcher chooses a set of requests B_t and a shared profile:

$$\Pi(B_t, t) = \text{Share}(\{\pi(r, t) : r \in B_t\})$$

The objective is to minimize latency, memory traffic, switching overhead, and quality risk:

$$\min_{B_t, \Pi} C_{\text{latency}} + \lambda C_{\text{traffic}} + \mu C_{\text{switch}} + \rho C_{\text{risk}}$$

subject to:

$$\text{quality loss} \leq \epsilon, \quad \text{deadline misses} \leq \delta, \quad \text{memory} \leq M.$$

This is a mixed precision problem because the precision allocation is no longer only inside the model. It is also across requests, queue slots, memory cache entries, and kernel paths.

2. Background: mixed precision methods and what they miss

2.1 Uniform quantization

Uniform post-training quantization (PTQ) and quantization-aware training (QAT) use a single global format, such as W8A8, W4A16, or NF4-style 4-bit weights. Uniform methods are attractive because storage layouts are simple, kernels are reusable, and deployment is predictable. GPTQ showed that one-shot weight quantization can push large generative transformers to 3 or 4 bits with strong accuracy and end-to-end speedups under suitable kernels. QLoRA popularized NF4 for memory-efficient finetuning, although it is mainly a training or adaptation method rather than a serving-time mixed-precision scheduler.

The limitation is that uniform precision pays the same bit budget for every prompt, layer, and token. It cannot selectively protect fragile layers or outlier-heavy inputs. If configured conservatively, it wastes bandwidth on easy requests. If configured aggressively, hard requests may suffer quality cliffs.

2.2 Static outlier-aware quantization

Outlier-aware static methods protect the components most likely to break under low precision. LLM.int8 isolates emergent outlier dimensions into a higher precision path while using int8 for most values. SmoothQuant migrates activation quantization difficulty into weights through an offline equivalent transformation so that W8A8 execution becomes more practical. AWQ observes that only a small fraction of salient weights may need protection, using activation statistics to identify them and scaling salient channels rather than relying on hardware-unfriendly arbitrary mixed precision.

These methods are powerful because they make quantization more robust while keeping the run-time path mostly regular. But their policies are primarily fixed after calibration. They protect expected outliers, not necessarily the prompt-specific or token-specific outliers that appear at run-time.

2.3 Static mixed precision

Static mixed precision allocates more bits to sensitive layers, groups, channels, or tensor components. HAWQ-V2 uses second-order information to select layer-wise precision and a Pareto frontier to automate bit selection. Modern LLM mixed-precision methods extend this idea with salience, gradients, Hessian approximations, low-rank residuals, channel-wise policies, or interaction-aware allocation.

Static mixed precision addresses a key weakness of uniform quantization: not all layers are equally sensitive. However, it assumes the sensitivity profile is stable enough to fix at deployment time. That assumption becomes weak when prompts and decoding steps induce different activation patterns.

2.4 Any-precision and nested representations

Any-precision methods store one model in a way that can realize multiple effective bit-widths. Any-Precision LLM overlays bit-width variants into a memory footprint comparable to a single high-bit model. AnyBCQ represents weights with binary bit-planes and scale factors, enabling precision expansion as more planes are enabled. MoBiQuant uses recursive residual slices so that 2-bit, 4-bit, 6-bit, and 8-bit behavior can be realized by activating more fixed-size slices.

These methods are important for dynamic batching because they provide a representation that a scheduler can control. Without a nested or bit-plane representation, switching precision usually means loading a different checkpoint or repacking weights, which is too expensive for per-request adaptation.

The remaining question is: who decides which precision to activate, and how can many requests share those decisions?

2.5 Runtime adaptive methods

The project deck centers on three runtime-adaptive papers:

- **QAQ**: query-adaptive mixed-precision quantization. It decomposes weights into bit-planes and uses a trainable router to select how many significant planes each request or block should use. It exposes a clear system bottleneck: on-demand loading can reduce GPU memory footprint but synchronous CPU-GPU transfers can erase latency gains.
- **DP-LLM**: dynamic layer-wise precision assignment. It observes that layer sensitivity changes across decoding steps and uses lightweight relative-error estimation plus thresholds to choose low or high precision per layer at runtime.
- **MoBiQuant**: token-adaptive any-precision quantization. It identifies precision-dependent outlier migration, then uses recursive residual bit slices and a token-aware router to activate extra slices for sensitive tokens.

These works show that dynamic precision is useful. They also show why serving-level coordination is missing. If QAO, DP-LLM, or MoBiQuant is applied independently to every request in a multi-request batch, the executor may see a chaotic mixture of bit-plane requirements, layer-wise decisions, and token-level slice masks.

2.6 Continuous batching and serving systems

Serving systems already recognize that autoregressive generation is a scheduling problem. ORCA introduced iteration-level scheduling, where the scheduler interacts with the execution engine at each generation iteration rather than waiting for whole requests to finish. Modern systems such as vLLM, Triton, and TensorRT-LLM implement continuous or in-flight batching so that finished requests can leave and new requests can enter at each step. vLLM additionally uses PagedAttention to manage KV cache memory with paging-like block allocation.

The missing intersection is precision. Current serving batchers mainly reason about arrival time, sequence length, prefill/decode phase, KV-cache blocks, LoRA adapters, parallelism, and sometimes priority. A precision-aware batcher would add a new compatibility dimension: can these requests share the same quantized weight pages, bit-planes, residual slices, and kernels?

3. Taxonomy and comparison

3.1 Method comparison

Family	Decision time	Granularity	Runtime signal	Main strength	Main weakness
Uniform PTQ/QAT	Offline	Whole model	Calibration set	Simple layout and kernels	Over-provisions easy components or harms fragile ones
Static outlier-aware	Offline	Channel, group, outlier path	Calibration activations	Robust static deployment	Cannot react to prompt-specific difficulty
Static mixed precision	Offline	Layer, group, channel	Hessian, salience, gradient, reconstruction loss	Better accuracy at low average bits	Fixed policy; no per-request or per-token adaptation
Any-precision representation	Offline plus runtime knob	Bit-plane or nested model	Target bit-width or budget	One model can support several precisions	Needs a policy and efficient switching
Query-adaptive precision	Runtime per request	Request, block	Prompt features, hidden states, outlier indicators	Avoids worst-case over-provisioning	Can cause bit-plane loading and profile fragmentation

Family	Decision time	Granularity	Runtime signal	Main strength	Main weakness
Layer-step adaptive precision	Runtime per decoding step	Layer	Relative error estimate	Captures changing layer sensitivity	Selector overhead and hard to batch if each request differs
Token-adaptive precision	Runtime per token	Token, slice	Token sensitivity and router score	Handles outlier migration	Fine-grained masks can be irregular for batched execution
Precision-aware dynamic batching	Runtime serving loop	Batch, microbatch, profile lane	Precision profile, queue state, deadlines, cache state	Makes adaptivity executable at scale	Scheduler complexity and accuracy-risk management

3.2 Why dynamic batching is different from prior mixed precision

The key difference is the optimization target. Prior mixed-precision work typically minimizes model error or latency for one request under a memory or bit budget. Precision-aware batching minimizes **system cost for a set of requests** under accuracy and QoS constraints.

This creates new design variables:

- Queueing delay versus profile compatibility.
- Batch size versus precision homogeneity.
- Shared profile safety versus per-request waste.
- Prefetch hit rate versus memory occupancy.
- Fallback sensitivity versus scheduling stability.
- Kernel specialization versus number of compiled profiles.

A local quantizer can say, “request A should use 3-bit in most blocks and 6-bit in late MLP layers.” A serving scheduler must decide whether request A should wait a few milliseconds for similar requests, run immediately with a less compatible batch, be upgraded to a safe shared profile, or be moved to a fallback lane.

3.3 Relationship to soft pruning

Dynamic mixed precision can be viewed as soft pruning. Hard pruning removes a component or skips a block. Soft pruning preserves the component but reduces its effective information budget. Dynamic batching extends soft pruning to the batch level:

- A low-budget batch is a soft-pruned execution path for easy requests.
- A medium profile batch spends extra bits only on shared sensitive layer groups.
- A fallback lane protects requests whose uncertainty signal suggests quality risk.

The advantage over hard pruning is that the model can preserve information rather than dropping it. The disadvantage is that lower-bit execution only helps if the system has efficient low-bit kernels and memory layouts.

4. Proposed research design: serving-aware dynamic batching

4.1 High-level architecture

The proposed architecture has five layers:

Incoming requests

|
v

Query and QoS analyzer

- prompt length, task type, entropy proxy, deadline, user priority

|
v

Precision profile predictor

- request budget
- layer/block precision vector
- uncertainty score
- expected bit-plane or slice demand

|
v

Precision-aware dynamic batcher

- queue window
- profile compatibility matching
- deadline-aware bucket assignment
- shared batch profile composition

|
v

Mixed-precision executor

- prefetch selected planes or slices
- run shared profile kernels
- use token fallback for risky requests

|
v

Feedback and trace store

- accuracy proxies, fallback rate, latency, traffic, cache hits

The scheduler sits above QAQ-like, DP-LLM-like, and MoBiQuant-like mechanisms. It does not replace them. It makes their decisions batch-compatible.

4.2 Precision profile representation

A precision profile should be compact enough for scheduling but expressive enough to predict execution cost. A practical profile can use layer groups rather than individual matrices. For a transformer block, the profile may split into:

- Attention QKV projections.
- Attention output projection.
- MLP gate/up projections.
- MLP down projection.
- Embedding and LM head.
- KV cache precision.

A profile can then be represented as:

$$\pi_r = (b_r, g_r, h_r, u_r, f_r)$$

where:

- b_r is average target bits.
- g_r is a short layer-group vector, for example [low, low, high-MLP, high-attn].
- h_r is a set of high-risk blocks.
- u_r is uncertainty.
- f_r is the fallback policy.

The profile should not be too detailed. If it contains a unique bit-width for every tensor and token, it will fragment the queue. Profile compression is part of the research problem.

4.3 Compatibility distance

Requests can be grouped using a compatibility distance:

$$D(i, j) = \alpha |b_i - b_j| + \beta (1 - J(H_i, H_j)) + \gamma |\ell_i - \ell_j| + \eta Q(i, j)$$

where:

- b_i and b_j are scalar precision budgets.
- $J(H_i, H_j)$ is Jaccard similarity between high-precision block sets.
- ℓ_i and ℓ_j are sequence or context lengths.
- $Q(i, j)$ penalizes incompatible QoS deadlines, adapter IDs, memory residency, or pre-fill/decode phase.

This formulation makes dynamic batching a multi-objective clustering problem. The distance should be cheap to compute and should predict the true execution penalty of batching two requests together.

4.4 Batch profile composition policies

Once a batch is formed, the executor needs one shared precision profile. Several policies are possible.

Max profile

$$P_{B,l} = \max_{r \in B} p_{r,l}$$

This is the safest policy. Every request receives at least as much precision as predicted. The weakness is over-provisioning: one hard request can upgrade a whole batch.

Mean or rounded profile

$$P_{B,l} = \text{round} \left(\frac{1}{|B|} \sum_{r \in B} p_{r,l} \right)$$

This reduces cost but can under-provision sensitive requests. It needs robust fallback.

Quantile profile

$$P_{B,l} = Q_\tau(\{p_{r,l} : r \in B\})$$

A quantile policy is a useful middle ground. With $\tau = 0.75$, most requests are protected without letting a single extreme request dominate. Requests above the quantile can use token-level fallback, local recomputation, or exit the batch into a higher precision lane.

Water-filling profile

The scheduler can allocate extra bits to the layer-request pairs with the largest expected marginal quality gain per byte:

$$\Delta_{r,l} = \frac{\text{risk reduction from one more bit}}{\text{extra traffic or latency}}$$

The batch receives additional bits where aggregated marginal gain is highest. This is more principled but requires calibrated risk estimates.

Lane split

The scheduler may split the queue into a small number of compiled lanes:

- Lane A: low budget, mostly easy requests.
- Lane B: medium budget, shared high-precision MLP or attention groups.
- Lane C: hard or uncertain fallback.

Lane split is often more hardware-friendly than trying to support arbitrary precision vectors.

4.5 Offline trace collection

The report’s proposed research should begin with traces, not kernels. The scheduler needs to know which profile differences matter. Four traces are essential:

1. **Query difficulty trace:** length, task type, prompt embedding, early hidden-state statistics, router score, entropy proxy, and observed fallback need.
2. **Layer relative-error trace:** estimated output difference between low-bit and high-bit weights for each layer group across decoding steps.
3. **Token outlier trace:** token positions or token types that repeatedly create high quantization error under different bit-widths.
4. **System trace:** HBM bytes, PCIe/NVLink transfers, kernel launches, queue delay, cache hits, CUDA graph reuse, and GPU utilization.

The goal is to discover stable patterns:

- Math prompts may need high precision in late MLP blocks.
- Long-context prompts may need higher attention or KV-cache precision.
- Code generation may stress different projection groups than summarization.
- Certain layer groups may switch together often enough to become one compiled profile.

4.6 Profile discovery and compilation

After trace collection, the system should build a small vocabulary of precision profiles. Good profiles have:

- High support: they occur often in real traffic.
- Low variance: their accuracy risk is predictable.
- Low switching cost: they reuse resident weight pages or compiled kernels.
- Hardware validity: their bit-widths align with supported kernels and layouts.

The compiler should turn these profiles into execution paths:

- Fused kernels for common low/high layer group patterns.
- CUDA graph capture for repeated decode shapes and profile lanes.
- Bit-major packing for bit-planes or residual slices.
- Prefetch plans for the next layer group or next decode step.
- Profile-specific memory residency rules.

The number of compiled profiles must be small. A profile library of 8 to 32 lanes may be much more practical than hundreds of arbitrary vectors.

4.7 Online scheduling policy

At runtime, the scheduler receives requests and assigns them to queues. A simple policy is:

For each incoming request r :

1. Compute scalar budget b_r and profile vector g_r .
2. Estimate uncertainty u_r and deadline d_r .
3. Find nearest compiled profile lane k using compatibility distance.
4. Insert r into lane queue k .
5. If lane queue reaches batch target, schedule it.
6. If r approaches deadline, schedule with best available compatible batch.
7. If u_r is high, assign r to fallback lane or require higher quantile profile.

During decoding, the same idea runs at iteration level. Completed requests leave. New requests enter. Requests whose risk changes can migrate to another lane, but migration should be limited to avoid thrashing.

4.8 Runtime fallback

Fallback is the safety valve that allows aggressive batching. Without fallback, the scheduler must use conservative max profiles. With fallback, most of the batch can run at a shared lower profile, while risky cases activate extra precision locally.

Useful fallback signals include:

- High next-token entropy.
- Small probability gap between top candidates.
- Large activation norm or outlier channel magnitude.
- High estimated low-bit versus high-bit relative error.
- Router uncertainty or profile mismatch score.
- Repeated low-confidence steps for the same request.

Fallback mechanisms include:

- Activate extra residual slices for the token.
- Recompute a layer group at higher precision.
- Upgrade the next decoding step's profile.
- Move the request to a fallback lane.
- Use high precision for prefill but dynamic precision for decode.

Fallback should be rare and predictable. If fallback becomes frequent, the scheduler is underestimating request difficulty or the profile vocabulary is too coarse.

5. Strengths of dynamic batching as mixed precision

5.1 It attacks the systems bottleneck that local quantizers expose

QAQ-like methods can reduce GPU memory footprint by loading only selected bit-planes, but on-demand loading can increase latency if transfers are synchronous. A precision-aware batcher can group requests that need the same planes, prefetch them once, and amortize transfer cost across the batch.

5.2 It improves cache and memory residency

If the scheduler knows that a lane will repeatedly use a particular block-level profile, it can keep those planes or residual slices resident in GPU memory. This is more effective than reacting to each request independently. It also enables a multi-level cache policy:

- Always resident: base low-bit slices and common high-risk layers.
- Profile resident: planes used by active lanes.
- Prefetch candidate: planes likely needed soon.
- CPU or host memory: rare fallback slices.

5.3 It reduces kernel and CUDA graph fragmentation

Dynamic precision can break kernel regularity. A profile-aware batcher constrains the number of precision paths. This makes it easier to use specialized kernels, graph capture, and fused operations. It also reduces the number of distinct kernels launched per token.

5.4 It aligns precision with QoS

Requests have different latency and quality requirements. Some users may prefer fast approximate answers; others may require high reliability. Dynamic batching can assign requests to precision lanes that match QoS classes while preserving throughput.

5.5 It prevents easy requests from being dragged into worst-case precision

Static quantization often protects hard cases by using a conservative bit-width for everyone. Precision-aware batching separates easy and hard requests. Easy requests can share low-bit execution paths. Hard requests can use medium or fallback lanes without forcing the entire traffic stream to high precision.

5.6 It creates a research bridge between quantization and serving

Mixed precision research often stops at model quality and kernel microbenchmarks. Serving research often assumes a fixed model precision. Dynamic batching forces the two communities to share metrics: accuracy, tail latency, memory traffic, cache residency, queueing delay, kernel support, and fallback risk.

6. Weaknesses and risks

6.1 Queueing delay can erase latency gains

Precision-aware grouping may require waiting for compatible requests. If traffic is low or profiles are too fragmented, the waiting cost can exceed the benefit of shared execution. This is especially dangerous for time-to-first-token (TTFT).

Solution: use strict queue deadlines and degrade gracefully. If no compatible requests arrive, run the request alone or in the closest available lane.

6.2 Profile prediction can be wrong

A query may look easy but become hard during generation. A low-bit profile can cause early token mistakes that change the future trajectory. This is more severe than a simple numeric error because autoregressive generation compounds mistakes.

Solution: use uncertainty-aware fallback, periodic high-precision checkpoints, and online feedback. Treat the predictor as a risk estimator, not an oracle.

6.3 Shared profiles can waste bits

A max shared profile is safe but can waste precision on easy requests. A mean profile is efficient but can under-provision hard ones. Quantile profiles require careful calibration.

Solution: split lanes by high-risk layers and use token-level fallback for the tail of the risk distribution.

6.4 Too many profiles create fragmentation

If the profile vocabulary is too large, each lane has low occupancy. If it is too small, the profiles are inaccurate. This is the central compression trade-off of the scheduler itself.

Solution: learn a small profile codebook from traces and merge profiles using execution cost, not just prediction similarity.

6.5 Fine-grained token masks can be hardware-unfriendly

Token-level routing is attractive algorithmically. But if every token activates a different set of slices, the kernel may need scatter-gather logic, predication, or multiple passes.

Solution: constrain token fallback to fixed slice increments and perform fallback in grouped microbatches. Another option is to use a shared low-bit main path plus sparse high-bit correction for risky tokens.

6.6 Interaction with KV cache is underdeveloped

Weight precision is only part of inference. Long context shifts the bottleneck to attention and KV-cache memory. Dynamic batching that ignores KV cache precision may optimize the wrong cost.

Solution: include KV-cache precision and page residency in the profile. The batcher should know whether a request is weight-bound, KV-bound, or transfer-bound.

6.7 Evaluation is harder than single-request quantization

A quantization paper can report perplexity, task accuracy, and tokens per second. A precision-aware batching paper must evaluate under realistic traffic traces, deadlines, variable output lengths, and multi-tenant QoS. The result may depend strongly on arrival rate.

Solution: build a trace-driven simulator and then validate key points in an integrated serving prototype.

7. Bottlenecks and suggested solutions

7.1 CPU-GPU and GPU-GPU transfer bottleneck

Problem. Bit-plane and residual-slice methods often keep only part of the representation resident in GPU memory. If high precision is requested unexpectedly, the system may need to transfer additional planes from CPU memory or another GPU. Synchronous transfer can dominate token latency.

Solutions.

- Use profile-aware prefetch. The scheduler should issue transfers before the layer needs them.
- Use double buffering and separate CUDA streams for transfer and compute.
- Keep base slices and frequent fallback slices resident.
- Use lane-level caching: if a lane is active, keep its profile pages hot.

- Predict several decoding steps ahead when possible, but cap speculation to avoid wasting memory.
- Prefer residual-slice layouts that allow monotonic precision upgrades without repacking.

7.2 Irregular memory layout and bit packing

Problem. Mixing 2-bit, 3-bit, 4-bit, 6-bit, and 8-bit tensors can create unaligned addresses and inefficient loads. Arbitrary layer or token precision can also break vectorized access.

Solutions.

- Store weights in bit-major or slice-major layout.
- Use fixed slice sizes, such as 2-bit residual slices, so higher precision is a sum of regular pieces.
- Align slice pages to cache-line and tensor-core-friendly boundaries.
- Restrict compiled profiles to hardware-supported bit groups.
- Use group-wise precision instead of element-wise precision unless there is dedicated hardware support.

7.3 Kernel switching and launch overhead

Problem. Dynamic precision may require launching different kernels for different bit-widths or layer groups. If the number of switches per token is high, launch overhead and synchronization can erase low-bit gains.

Solutions.

- Compile a small profile library and reuse kernels.
- Use CUDA graphs for common decode shapes and profile lanes.
- Fuse low/high precision paths when they frequently co-occur.
- Use grouped GEMM for requests sharing precision and shape.
- Batch fallback tokens together instead of invoking tiny per-token kernels.

7.4 Selector overhead

Problem. A runtime selector that estimates relative error or token sensitivity can itself become expensive. If it requires extra GEMV/GEMM operations, the overhead may be larger than the saved precision cost.

Solutions.

- Use cheap features: activation norms, low-dimensional random projections, entropy, top-k probability gap, or cached prompt embeddings.
- Run selector computation asynchronously when possible.
- Skip detailed routing when the request matches a high-confidence compiled profile.
- Use a two-stage selector: cheap coarse classifier first, detailed selector only for uncertain cases.
- Quantize or distill the selector itself.

7.5 Queue fragmentation

Problem. A scheduler that distinguishes too many precision profiles will create many small batches. This reduces GPU occupancy and increases per-request overhead.

Solutions.

- Cluster profiles into a small codebook.
- Use deadlines to bound waiting.
- Merge profiles if their execution cost difference is smaller than the queueing cost.

- Use max or quantile profile composition within each lane.
- Share profiles across adjacent bit budgets, such as 3.5-bit and 3.75-bit, when risk is low.

7.6 Accuracy cliff and autoregressive error propagation

Problem. A single low-precision mistake can push generation into a different trajectory. Perplexity may not fully capture this failure mode.

Solutions.

- Use high precision during prefill or early decoding for hard prompts.
- Use periodic high-precision anchor steps.
- Trigger fallback when entropy or probability gap suggests ambiguity.
- Evaluate exact-match, reasoning, code, and safety-sensitive tasks, not only perplexity.
- Track divergence from a high-precision reference on sampled traces.

7.7 Prefill/decode asymmetry

Problem. Prefill is compute-heavy and often uses large matrix-matrix operations, while decode is usually more memory-bound and iterative. A profile that helps decode may not help prefill.

Solutions.

- Use separate profile policies for prefill and decode.
- Use high or medium precision for prefill if prompt encoding errors are expensive.
- Apply dynamic precision more aggressively during decode.
- Use chunked prefill and profile-aware chunk scheduling for long prompts.

7.8 KV-cache precision and memory pressure

Problem. Dynamic weight precision does not solve KV-cache growth. Long-context workloads can become KV-bound, and mixed precision of weights may produce limited benefit.

Solutions.

- Add KV-cache precision to the profile.
- Use page-level KV cache metadata, including precision and residency.
- Apply higher KV precision to attention-sensitive or long-range dependency tokens.
- Co-design with PagedAttention-like memory managers.
- Measure HBM bytes per generated token, not only model weight bytes.

7.9 Multi-GPU communication

Problem. In tensor-parallel or pipeline-parallel inference, precision decisions affect communication volume and synchronization. A request that upgrades precision in one stage can create imbalance.

Solutions.

- Make profile lanes visible to all parallel ranks.
- Use stage-aware precision profiles, with different policies for attention, MLP, and communication-heavy layers.
- Prefetch slices across GPUs using NVLink-aware scheduling.
- Add communication cost into compatibility distance.

7.10 Lack of benchmarks

Problem. There is no standard benchmark for precision-aware dynamic batching. Single-request quantization benchmarks do not reveal queueing and profile fragmentation.

Solutions.

- Use trace-driven evaluation with variable arrival rates, prompt lengths, output lengths, and task classes.
- Report P50/P90/P99 latency, TTFT, TPOT, throughput, memory traffic, transfer stalls, profile switches, and fallback rate.
- Include multi-tenant QoS and deadline misses.
- Publish synthetic and real traffic traces with precision labels or proxy labels.

8. Evaluation plan

8.1 Baselines

A strong evaluation should compare against:

1. **FP16/BF16 static serving**: quality upper bound and memory-heavy baseline.
2. **Uniform quantized serving**: W8A8, W4A16, GPTQ, AWQ, or relevant production quantization.
3. **Static mixed precision**: HAWQ-V2-like or salience-based layer/group allocation.
4. **Any-precision without precision-aware batching**: request-level precision selection but ordinary batching.
5. **QAQ-like query routing without asynchronous batching**: shows transfer and profile-switch cost.
6. **DP-LLM-like dynamic layer selection without cross-request coordination**: shows selector benefit and batching difficulty.
7. **MoBiQuant-like token routing without serving-level profile lanes**: shows token elasticity but possible irregularity.
8. **Current serving engine continuous batching**: vLLM, Triton, or TensorRT-LLM with a fixed quantized model.
9. **Multi-checkpoint routing**: route requests to separate static quantized engines, useful as a practical but memory-expensive baseline.

8.2 Metrics

Quality metrics

- Perplexity on WikiText2, C4, and domain-specific corpora.
- Task accuracy on reasoning, math, coding, summarization, retrieval-augmented QA, and instruction-following tasks.
- Divergence from FP16 reference output.
- Safety and refusal behavior under quantization, if the model is used in safety-sensitive settings.

Serving metrics

- TTFT: time to first token.
- TPOT: time per output token.
- End-to-end latency at P50, P90, P95, and P99.
- Throughput in tokens/sec and requests/sec.
- Deadline miss rate.
- GPU utilization and SM occupancy.
- HBM bytes/token and memory bandwidth utilization.
- PCIe/NVLink transfer bytes/token.
- Kernel launches/token.
- CUDA graph reuse rate.
- Profile switch count.

- Prefetch hit rate.
- Fallback rate and fallback latency.
- Average queueing delay introduced by profile matching.

Scheduler metrics

- Batch compatibility score.
- Profile entropy across active requests.
- Lane occupancy.
- Misroute rate: requests that needed fallback after being classified as easy.
- Over-precision rate: extra bits spent beyond request-level predicted need.
- Under-precision risk: estimated or observed degradation due to shared lower profile.

8.3 Workloads

The evaluation should use a mixture of traffic classes:

- Short chat prompts.
- Long-context summarization.
- Code generation.
- Math and reasoning prompts.
- Retrieval-augmented prompts with long context.
- Multi-turn conversations with shared prefix cache.
- Mixed QoS workloads with fast and accurate service tiers.

Traffic should vary arrival rate from low load to saturation. Precision-aware batching may look best near medium or high load, where enough compatible requests exist. Under low load, the scheduler must avoid waiting too long.

8.4 Ablations

Important ablations include:

- No profile matching: ordinary dynamic batching.
- Scalar budget only versus block-vector profile.
- Max profile versus mean profile versus quantile profile.
- No fallback versus token-level fallback versus request migration.
- Synchronous loading versus asynchronous prefetch.
- Large profile vocabulary versus small profile vocabulary.
- Prefill-only profiling versus decode-only profiling versus phase-specific profiles.
- Weight-only precision versus weight plus KV-cache precision.
- Selector always on versus compiled-profile fast path.

8.5 Hypotheses

The research can be organized around testable hypotheses:

- **H1:** Profile-aware batching reduces bit-plane or slice transfer bytes per token compared with per-request dynamic precision under ordinary continuous batching.
- **H2:** A small compiled profile codebook captures most of the quality benefit of fine-grained dynamic precision while reducing kernel switches.
- **H3:** Quantile profile sharing plus token fallback gives a better latency-quality frontier than max profile sharing.
- **H4:** Asynchronous prefetch converts dynamic precision from latency overhead into memory-capacity benefit for QAQ-like bit-plane representations.
- **H5:** Profile-aware batching improves P99 latency under mixed traffic by isolating hard or uncertain requests into fallback lanes instead of letting them upgrade every batch.

9. Implementation roadmap

9.1 Stage 0: Trace-driven simulator

Start with simulation. Use calibration and serving traces to model:

- Request arrival times.
- Prompt and output length distribution.
- Predicted precision profiles.
- Transfer cost for each profile.
- Kernel cost for each precision lane.
- Accuracy risk and fallback probability.

A simulator allows fast exploration of scheduling policies before kernel implementation.

9.2 Stage 1: Multi-engine prototype

A practical first prototype can route requests to several static quantized engines, such as 3-bit, 4-bit, and 8-bit variants. This does not provide true shared bit-plane execution, but it tests whether profile-aware queueing helps under real serving workloads.

This stage answers:

- Does query difficulty prediction correlate with needed precision?
- Does profile-aware queueing reduce tail latency or harm TTFT?
- How many precision lanes are useful?
- How sensitive are results to traffic load?

9.3 Stage 2: Nested representation executor

Next, implement a nested representation such as bit-planes or residual slices. The executor should support:

- Base low-bit weights resident on GPU.
- Optional slices for higher precision.
- Profile-specific prefetch.
- Shared block-level profile per batch.
- Token-level fallback as a second pass or grouped microbatch.

This stage tests the central systems claim: adaptive precision only works at scale if movement and switching are amortized across compatible batches.

9.4 Stage 3: Continuous precision-aware batching

Integrate with an iteration-level scheduler. At every decode step:

- Remove completed requests.
- Admit new requests.
- Update uncertainty and profile state.
- Keep requests in lanes unless migration is necessary.
- Compose shared batch profiles.
- Schedule prefetch for future layers or steps.

This stage should report TTFT, TPOT, P99 latency, throughput, profile switches, fallback rate, and traffic.

9.5 Stage 4: Compiler and hardware co-design

Once profiles are stable, compile common paths:

- Low-bit lane kernels.
- Medium lane kernels with selected high-precision layer groups.
- Fallback lane kernels.
- CUDA graphs for common batch sizes and decode shapes.
- Layout transforms for bit-major or slice-major storage.

The final research contribution can be framed as an algorithm-system-compiler co-design.

10. Detailed design choices

10.1 Profile codebook construction

A profile codebook can be learned from traces using clustering. The distance metric should combine quality and execution cost. A profile that looks different but uses the same resident slices may be cheap to merge. A profile that differs by one expensive high-traffic layer may be costly to merge.

A practical construction process:

1. Collect fine-grained ideal profiles on calibration traces.
2. Convert fine profiles to coarse layer-group features.
3. Cluster by execution-cost-aware distance.
4. For each cluster, choose a representative profile using quantile or water-filling.
5. Validate quality risk on held-out prompts.
6. Compile only profiles that pass support and risk thresholds.

10.2 Request difficulty prediction

The predictor can use:

- Prompt length.
- Prompt embedding or task classifier.
- Early hidden-state norms.
- Activation outlier statistics from the first few layers.
- Entropy from a small draft model.
- User QoS tier.
- Historical behavior for similar prompts.

The predictor does not need perfect bit assignments. It only needs to place requests in a lane where shared execution is safe and efficient. This is a less demanding target than exact per-layer routing.

10.3 Layer sensitivity prediction

For DP-LLM-like behavior, layer sensitivity can be approximated by relative error between low-bit and high-bit outputs. Exact computation is too expensive, so the scheduler should use approximate indicators:

- Activation norm per layer group.
- Random projection of input vectors.
- Learned small regressors.
- Offline thresholds per layer group.
- Historical fallback frequency.

The batcher can use coarse layer groups first, then rely on executor fallback for local corrections.

10.4 Token fallback design

Token fallback should be designed to preserve regularity:

- Keep the main batch path shared.
- Mark risky tokens during or after the low-bit pass.
- Group risky tokens by fallback slice count.
- Run one or a few correction kernels.
- Update future request profile if fallback repeats.

This avoids fully arbitrary token-level routing inside every kernel.

10.5 Memory policy

A memory policy should classify pages or slices by expected reuse:

Memory class	Example	Policy
Always resident	Base 2-bit or 4-bit weights	Never evict during serving window
Lane resident	Frequent high-bit slices for active profile lanes	Keep while lane occupancy is high
Prefetch	Slices predicted for next step or block	Load asynchronously
Fallback cache	Rare high-risk slices	Keep if recent fallback rate is high
Cold storage	Rare profiles	CPU memory, SSD, or remote memory if applicable

The scheduler should expose future profile demand to the memory manager. Otherwise, the memory manager only reacts after a stall occurs.

11. How this compares with other approaches

11.1 Compared with uniform PTQ

Uniform PTQ is simpler, more robust, and easier to deploy. Precision-aware batching is more complex but can exploit input heterogeneity. It should be used when traffic contains a mix of easy and hard prompts and when the hardware can efficiently switch among a small set of profiles.

Uniform PTQ is preferable when:

- Traffic is homogeneous.
- Latency budget is extremely tight.
- The serving stack lacks low-bit dynamic kernels.
- Accuracy risk from misrouting is unacceptable.

Precision-aware batching is preferable when:

- There is high prompt heterogeneity.
- GPU memory is tight.
- Load is high enough for compatible grouping.
- A small profile codebook captures most dynamic behavior.

11.2 Compared with static outlier-aware methods

Outlier-aware methods such as LLM.int8, SmoothQuant, and AWQ are excellent for removing known quantization hazards. Precision-aware batching should not replace them. It should build

on them. Static transforms can make the base low-bit path stable; dynamic batching decides when extra bits are worth spending.

In other words:

- Static outlier-aware quantization improves the default path.
- Dynamic batching manages per-request and per-batch deviations from the default path.

11.3 Compared with static mixed precision

Static mixed precision spends bits based on average sensitivity. Dynamic batching spends bits based on request-specific and batch-specific sensitivity. Static mixed precision is easier to compile. Dynamic batching can exploit runtime variation but needs a scheduler, predictors, and fallback.

A good system may combine them:

- Use static mixed precision to define a robust base profile.
- Use dynamic batching to select among a few upgrades.
- Use token fallback for rare residual risk.

11.4 Compared with QAQ

QAQ makes precision query-adaptive by selecting bit-planes based on query difficulty. Dynamic batching adds a serving-level question: which queries should share bit-plane loading and execution?

QAQ's bottleneck is memory movement. Dynamic batching directly targets that bottleneck through grouping, prefetch, and lane-level caching.

11.5 Compared with DP-LLM

DP-LLM captures token-step changes in layer sensitivity. Dynamic batching asks whether those per-layer decisions can be made compatible across requests. A direct integration could use DP-LLM's thresholds to generate a layer-group profile, then batch requests with similar high-risk layers.

The trade-off is granularity. Full per-layer DP-LLM routing may be too fragmented for serving. A profile-aware scheduler may need to coarsen DP-LLM decisions into layer groups or compiled lanes.

11.6 Compared with MoBiQuant

MoBiQuant's token-aware residual slices are a strong fit for fallback. The batch can run a shared low or medium precision path, while sensitive tokens activate additional slices. Dynamic batching uses MoBiQuant-like token routing as the local safety mechanism that allows coarse shared profiles.

The risk is irregular token masks. The solution is to group fallback tokens and keep slice increments fixed.

11.7 Compared with ordinary continuous batching

Ordinary continuous batching optimizes slot utilization. Precision-aware continuous batching optimizes slot utilization plus precision-path compatibility. It is a strict generalization if implemented with a fallback to ordinary scheduling when profiles are unavailable or traffic is sparse.

12. Future research directions

12.1 Precision-aware continuous batching in production engines

The next step is to integrate precision profiles into engines like vLLM, Triton, or TensorRT-LLM. The scheduler should treat precision lanes similarly to how current systems treat adapters, KV-cache blocks, or prefill/decode phases.

Research questions:

- What is the minimal API between quantizer and scheduler?
- Can a quantizer expose a compact profile without revealing implementation details?
- Can CUDA graphs be captured per precision lane?
- How should profile migration be handled at decode time?

12.2 Joint weight and KV-cache precision

Long-context serving will increasingly be KV-cache bound. Dynamic batching should include KV-cache precision in its profile. A request with easy weights but long-context attention sensitivity may require high KV precision rather than high weight precision.

Research questions:

- Which tokens in the KV cache deserve higher precision?
- Can KV pages be mixed precision without fragmenting PagedAttention?
- Can attention score uncertainty trigger KV precision upgrades?

12.3 Disaggregated prefill/decode scheduling

Modern serving systems increasingly separate prefill and decode. Precision-aware batching should exploit phase differences:

- Prefill: group by prompt length and high-level profile, possibly use medium/high precision for robust context encoding.
- Decode: use fine-grained profile lanes and token fallback.

Research questions:

- Should query difficulty be predicted after prefill rather than before?
- Can prefill output statistics improve decode lane assignment?
- How should bit-plane prefetch be coordinated across prefill and decode GPUs?

12.4 Safety-aware dynamic precision

Low precision can change refusal behavior, calibration, or reasoning reliability. Safety-critical prompts may need stricter fallback policies.

Research questions:

- Can safety classifiers inform precision budgets?
- Which safety behaviors are most sensitive to low-bit quantization?
- Can precision upgrades be used as a reliability mechanism for high-stakes responses?

12.5 Multi-tenant fairness and pricing

Precision is a resource that can be priced and scheduled. A production system may offer fast low-cost lanes and high-accuracy lanes.

Research questions:

- How should precision budget map to user-facing service tiers?

- Can the scheduler guarantee fairness across tiers?
- How should it prevent hard requests from starving low-budget traffic?

12.6 Hardware support for nested precision

Dynamic batching becomes much more practical if hardware supports bit-plane or residual-slice execution directly.

Research directions:

- Tensor-core support for common sub-byte formats.
- Efficient bit-serial or bit-parallel operations.
- Native support for monotonic precision expansion.
- Hardware prefetch hints based on precision profiles.
- On-chip caches aware of bit-plane residency.

12.7 Learned schedulers with safety constraints

A learned scheduler can adapt to traffic patterns, but it must obey deadlines and quality constraints. Reinforcement learning or contextual bandits may be useful, but the action space must be constrained by safe fallback policies.

Research questions:

- Can a scheduler learn when waiting for compatibility is worth it?
- Can it learn profile merging rules from measured kernel costs?
- Can it provide worst-case guarantees for deadlines and accuracy risk?

12.8 Standard benchmark for runtime mixed precision serving

The field needs a benchmark that combines model quality, dynamic traffic, and system metrics. A useful benchmark should include:

- Prompt and output length traces.
- Task labels and difficulty labels.
- Reference high-precision outputs.
- Precision profile traces or proxy labels.
- Serving load regimes.
- Metrics for quality, latency, traffic, and fallback.

Without this, papers may optimize single-request metrics that do not translate to serving.

13. Recommended thesis direction

A focused thesis could be:

Precision-aware dynamic batching for runtime adaptive mixed-precision LLM inference. The system predicts compact precision profiles, groups compatible requests into continuous batching lanes, executes a shared block-level precision plan, and uses token-level fallback to preserve quality.

The core contributions can be:

1. **Profile abstraction:** a compact representation connecting query difficulty, layer sensitivity, token fallback, and memory plan.
2. **Scheduling policy:** deadline-aware grouping by precision compatibility, not only arrival time or length.
3. **Profile sharing rule:** max, quantile, or water-filling precision composition with fallback.

4. **Memory and prefetch co-design:** asynchronous loading of bit-planes or residual slices based on lane demand.
5. **Evaluation methodology:** trace-driven metrics for accuracy, latency, traffic, profile fragmentation, and fallback.

The project should avoid trying to solve every granularity at once. A strong first version can use:

- Query-level budget predictor.
- Block-group precision vectors.
- Three to eight compiled lanes.
- Quantile profile sharing.
- Token-level fallback only for high-risk tokens.
- Trace-driven simulation plus a serving prototype.

14. Conclusion

Dynamic batching is a natural main focus for runtime adaptive mixed precision because it addresses the gap between algorithmic adaptivity and deployable inference. Query-level, layer-level, and token-level methods show that precision demand is input-dependent. But serving systems run many inputs together, and independent precision decisions can create irregular memory movement, fragmented kernels, and poor batch utilization.

The research opportunity is to make precision decisions batch-compatible. A precision-aware scheduler can group requests by predicted precision profile, select a shared batch plan, prefetch the required bit-planes or residual slices, and rely on token-level fallback for uncertain cases. This design preserves the key benefit of mixed precision - spending bits only where they matter - while respecting the hardware reality that GPUs reward regular, shared execution paths.

The strongest future work will likely be an algorithm-system co-design: a compact profile predictor, a small compiled profile vocabulary, an online deadline-aware scheduler, an asynchronous memory manager, and a safe fallback mechanism. If successful, dynamic batching will turn mixed precision from an offline compression technique into a serving-time resource allocation framework.

References

1. Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale." NeurIPS 2022. <https://arxiv.org/abs/2208.07339>
2. Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models." ICML 2023. <https://arxiv.org/abs/2211.10438>
3. Ji Lin et al. "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration." MLSys 2024. <https://arxiv.org/abs/2306.00978>
4. Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers." ICLR 2023. <https://arxiv.org/abs/2210.17323>
5. Zhen Dong et al. "HAWQ-V2: Hessian Aware trace-Weighted Quantization of Neural Networks." NeurIPS 2020. <https://arxiv.org/abs/1911.03852>
6. Yeonhong Park, Jake Hyun, SangLyul Cho, Bonggeun Sim, and Jae W. Lee. "Any-Precision LLM: Low-Cost Deployment of Multiple, Different-Sized LLMs." ICML 2024. <https://arxiv.org/abs/2402.10517>
7. Sangwoo Kwon, Seong Hoon Seo, Jae W. Lee, and Yeonhong Park. "DP-LLM: Runtime Model Adaptation with Dynamic Layer-wise Precision Assignment." NeurIPS 2025. <https://arxiv.org/abs/2508.06041>
8. S. Li et al. "QAQ: Query-adaptive Mixed-precision Quantization for Large Language Models." NeurIPS 2025 MLForSys Workshop. <https://openreview.net/forum?id=dpHfDasG44>

9. Dongwei Wang et al. "MoBiQuant: Mixture-of-Bits Quantization for Token-Adaptive Any-Precision LLM." arXiv, 2026. <https://arxiv.org/abs/2602.20191>
10. Gunho Park et al. "AnyBCQ: Hardware Efficient Flexible Binary-Coded Quantization for Multi-Precision LLMs." arXiv, 2025. <https://arxiv.org/abs/2510.10467>
11. Gyeong-In Yu et al. "Orca: A Distributed Serving System for Transformer-Based Generative Models." OSDI 2022. <https://www.usenix.org/conference/osdi22/presentation/yu>
12. Woosuk Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP 2023. <https://arxiv.org/abs/2309.06180>
13. NVIDIA Triton Inference Server documentation. "Batchers: Continuous/Inflight Batching with Iterative Sequences." https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/user_guide/batcher.html
14. NVIDIA TensorRT-LLM documentation. "Overview: In-Flight Batching, Paged Attention, Quantization." <https://nvidia.github.io/TensorRT-LLM/overview.html>
15. vLLM documentation. "Quantization" and "Welcome to vLLM." <https://docs.vllm.ai/en/latest/features/quantization/> and <https://docs.vllm.ai/>
16. Mariam Rakka et al. "Mixed-Precision Quantization for Language Models: Techniques and Prospects." arXiv, 2025. <https://arxiv.org/abs/2510.16805>